Les tuples et les ensembles

MAZZA AZZOUZ

CPGE NADOR
ECT-ECS
mazza8azzouz@gmail.com

25 octobre 2023

Les tuples sont des séquences qui fonctionnent comme des listes, la seule différence est qu'elles sont immuables, ce qui signifie que nous ne pouvons pas ajouter, supprimer ou modifier ses éléments une fois créés. Cela les rend également super efficaces par rapport à la liste. Les tuples sont utilisés pour stocker une liste d'éléments qui ne change pas.

Les valeurs d'un tuple sont syntaxiquement séparées par des «virgules». Bien que cela ne soit pas nécessaire, il est plus courant de définir un tuple en fermant la séquence de valeurs entre parenthèses. Cela aide à comprendre plus facilement les tuples Python.

En Python, les tuples sont créés en plaçant une séquence de valeurs séparées par des virgules avec ou sans parenthèses ().

```
1  # creation d'un tuple vide
2  tuple1=()
3  print(tuple1)
4
5  #tuple avec 3 elements de type chaine
6  tuple2=("Fes","Marrakech", "Errachidia")
7  print(tuple2)
8
9  #tuple avec un seul element
10 tuple3 =(4,)
11 print(tuple3)
12
13 #tuple sans parentheses
14 tuple4 = 3,5,7,10
15 print(tuple4)
```

Nous pouvons également utiliser le constructeur i.e tuple(), qui accepte tout type de séquence ou d'objet itératif.

```
1  # tuple de la chaine
2  t1 = tuple("abcd")
3  print(t1)
4
5  #tuple de range
6  t2 = tuple(range(1, 10))
7  print(t2)
8
9  # tuple de la liste
10 t3 = tuple([1,2,3,4])
11 print(t3)
```

Les tuples vous permettent d'attribuer des valeurs à plusieurs variables à la fois.

Le nombre de variables (à gauche) et le nombre d'éléments dans le tuple (à droite) doivent correspondre, sinon vous obtiendrez une erreur.

```
1 prenom,age=("Ismail",25)
2 print("Prenom : ",prenom)
3 print("Age : ",age)
```

```
Prénom : Ismail
Age : 25
```

Un tuple est essentiellement une liste immuable. En conséquence, la plupart des opérations pouvant être effectuées sur une liste sont également valables pour des tuples. Voici quelques exemples de telles opérations :

- Accédez à un seul élément ou à des tranches d'éléments à l'aide de l'opérateur [].
- Les fonctions intégrées telles que max(), min(), sum() sont valides sur un tuple.
- opérateurs d'appartenance in et not in
- Opérateurs de comparaison pour comparer les tuples.
- Opérateurs + et *.
- boucle for pour parcourir les éléments.
- ...

Les seules opérations que tuple ne supporte pas sont celles qui modifient la liste elle-même. Par conséquent, les méthodes telles que append(), insert(), remove(), reverse() et sort() ne fonctionnent pas avec les tuples.

```
1  tuple1=(2,4,5,7,19)
2  tuple2=(5,9,5)
3
4  # afficher l'element 4 de tuple1
5  print("4eme element : ",tuple1[3])
6
7  # afficher les 3 premiers elements
8  print("les 3 premiers elements sont : ",tuple1[:3])
9
10 # afficher le max de tuple1
11 print("max : ",max(tuple1))
12
13 # afficher la somme des elements de tuple1
14 print("somme : ",sum(tuple1))
15
16 tuple3=tuple1 + tuple2
17 print("concatenation de 2 tuples : ",tuple3)
18
19 # repeter un tuple 2 fois
20 tuple4=tuple1 * 2
21 print("double : ", tuple4)
22
23 #parcourir un tuple
24 for elem in tuple1:
25     print(elem,end=' , ')
```

Un ensemble est une collection non ordonnée d'éléments. Chaque élément est unique (pas de doublons) et doit être immuable (ce qui ne peut pas être changé). Cependant, l'ensemble lui-même est mutable. Nous pouvons ajouter ou supprimer des éléments.

Vous pouvez penser à des ensembles comme une liste. Cependant, ils diffèrent d'une liste des manières suivantes :

- Chaque élément à l'intérieur d'un ensemble doit être unique.
- Les éléments d'un ensemble ne sont pas stockés dans aucun ordre particulier.

Si votre application ne se soucie pas de la façon dont les éléments sont stockés, utilisez des ensembles plutôt que des listes, car en ce qui concerne la manipulation des éléments, un ensemble est beaucoup plus efficace qu'une liste.

Les ensembles peuvent être utilisés pour effectuer des opérations d'ensembles mathématiques telles que l'union, l'intersection, la différence symétrique, etc.

Un ensemble est créé en plaçant tous les éléments (éléments) entre accolades {}, séparés par une virgule ou en utilisant la fonction intégrée set().

Il peut avoir n'importe quel nombre d'éléments et ils peuvent être de types différents (int, float, tuple, string, etc.). Mais un ensemble ne peut pas avoir un élément mutable, comme une liste, un ensemble ou un dictionnaire.

```
1  # creation d'un ensemble de 4 elements
2  e1={1,3,5,7}
3  print("le type de e1 est : ",type(e1))
4  print("les elements de e1 sont : ",e1)
5
6  e2={"Ibrahim", 1.78, 32 }
7  print("e2 : ",e2)
8
9  # ensemble avec doublons
10 e3 = {3, 5, 7, 2, 3, 5}
11 print("e3 : ", e3)
```

Nous pouvons également utiliser la fonction intégrée set() pour créer des ensembles. Voici quelques exemples :

```
# creer un ensemble a partir d'un ensemble
e1 = set({77, 23, 91, 271})
print("e1 : ",e1)

#creer un ensemble a partir d'une chaine
e2 = set("123abc")
print("e2 : ",e2)

# creer un ensemble a partir d'une liste
e3 = set(['Mohamed', 'Alaoui', 'Fes', 29, 1.78])
print("e3 : ",e3)

#creer un ensemble a partir d'un tuple
e4 = set(("Fes","Marrakech", "Errachidia"))
print("e4 : ",e4)
```

Modifier un ensemble

Les ensembles sont mutables. Mais comme ils ne sont pas ordonnés, l'indexation n'a pas de sens.

Nous ne pouvons pas accéder à un élément d'un ensemble ni le modifier à l'aide de l'indexation ou du découpage en tranches. Set ne le supporte pas.

Nous pouvons ajouter un seul élément à l'aide de la méthode add() et plusieurs éléments à l'aide de la méthode update(). La méthode update() peut prendre pour argument des tuples, des listes, des chaînes ou d'autres ensembles. Dans tous les cas, les doublons sont évités.

```
1  e={1,3}
2
3  # ajouter un element
4  e.add(5)
5  print("e : ",e)
6
7  # ajouter une liste
8  e.update([9,4,7])
9  print("e : ",e)
10
11 # ajouter une liste et un ensemble
12 e.update([50,51],{100,200})
13 print("e : ",e)
```

Supprimer un élément d'un ensemble

Un élément particulier peut être supprimé de la série à l'aide des méthodes, discard() et remove().

La seule différence entre les deux est que, tout en utilisant discard() si l'élément n'existe pas dans l'ensemble, il reste inchangé. Mais remove() lève une erreur dans une telle condition.

```
1  e = {1, 3, 6, 7}
2
3  e.discard(3)
4  print("e : ", e)
5
6  e.remove(7)
7  print("e : ", e)
8
9  e.discard(9)
10 print("e : ", e)
11
12 e.remove(9)
13 print("e : ", e)
```



De même, nous pouvons supprimer et retourner un élément en utilisant la méthode pop().

Les ensembles n'étant pas ordonnés, il n'y a aucun moyen de déterminer quel élément sera supprimé. C'est complètement arbitraire.

Nous pouvons également supprimer tous les éléments d'un ensemble en utilisant clear().

```
1 e={2, 4, 90, 100, 30, 32, 1}
2 val=e.pop()
3 print("val : ",val)
4 print("e : ",e)
5
6 e.clear()
7 print("e : ",e)
```

Sous-ensemble et Sur-ensemble

L'ensemble B est un sous-ensemble de A, si tous les éléments de l'ensemble B sont également les éléments de l'ensemble A. Par exemple :

```
A = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
B = {1, 2, 3}
```

A est un ensemble de 10 premiers nombres naturels et B est un ensemble de 3 premiers nombres naturels. Tous les éléments de l'ensemble B sont également les éléments de l'ensemble B. Par conséquent, B est un sous-ensemble de A. En d'autres termes, nous pouvons également dire que l'ensemble A est un sur-ensemble de B.

Nous pouvons tester si un ensemble est un sous-ensemble ou un sur-ensemble d'un autre ensemble à l'aide des méthodes issubset() et issuperset().

```
# tester si B est un sous-ensemble de A
if B.issubset(A):
    print("B est un sous-ensemble de A")

# tester si A est un sur-ensemble de B
if A.issuperset(B):
    print("A est un sur-ensemble de B")
```

Comparer des ensembles

```
1  A = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2  B = {1, 2, 3}
3
4  if B < A:
5      print("B est un sous-ensemble de A")
6
7  if A > B:
8      print("A est un sur-ensemble de B")
9
10 if A == B:
11     print("A et B sont identiques")
12
13 if A>=B:
14     print("A est sur-ensemble ou egal a B")
```

Supposons que nous avons deux ensembles A et B, l'union de A et B est un ensemble composé de tous les éléments de A et tous les éléments de B. Les éléments en double ne seront inclus qu'une seule fois. En mathématiques, nous utilisons le symbole ∪ pour désigner l'union. Symboliquement, nous écrivons A union B comme A ∪ B.

Pour effectuer une opération d'union en Python, nous pouvons utiliser la méthode union() ou | opérateur.

L'intersection des ensembles A et B est un ensemble composé de tous les éléments communs à A et B. Le symbole ∩ indique l'intersection. Symboliquement, une intersection B s'écrit A ∩ B.

Pour effectuer une opération d'intersection en Python, nous utilisons la méthode intersection() ou l'opérateur &

Union et intersection

```
1 A = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 B = {1, 2, 3, 20, 34}
3
4 C=A.union(B)
5 print("Union : ", C)
6
7 C=A.intersection(B)
8 print("Intersection : ", C)
```

La différence des ensembles A et B est un ensemble d'éléments qui contient tous les éléments de l'ensemble A mais pas de B. Comme d'habitude, nous utilisons le symbole - pour désigner l'opération de différence. Symboliquement, A moins B s'écrit A - B.

En Python, nous utilisons la méthode difference() ou l'opérateur - pour effectuer la différence définie.

La différence symétrique de deux ensembles A et B est un ensemble constitué d'éléments qui sont dans un ensemble mais pas dans les deux. Le symbole Δ indique une différence symétrique.

En Python, nous utilisons la méthode symmetric_difference() ou l'opérateur^ pour effectuer cette opération.

Différence et différence symétrique

```
1 A = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10}
2 B = {1, 2, 3, 20, 34}
3
4 C=A.difference(B)
5 print("A - B : ", C)
6
7 C=A.symmetric_difference(B)
8 print("Difference symetrique : ", C)
```